3C

DESARROLLO DE SOFTWARE

Desarrollo de aplicaciones de Software para la solución de problemas

Saber, valorar, elegir

PADRE ARRUPE
Colegio Español

DOCENTE: Daniel Sosa

COLEGIO ESPAÑOL PADRE ARRUPE

Colegio internacional.

Tercer Año de Bachillerato de Software

Desarrollo de aplicaciones de Software para la solución de problemas

Periodo 1.

GUÍA 1 – Introducción a Microservicios con .NET

OBJETIVOS DE LA PRÁCTICA

1. **Comprender el concepto de microservicios**
   - Identificar las diferencias entre aplicaciones monolíticas y arquitecturas basadas en microservicios.
   - Reconocer las ventajas y desventajas de este enfoque.
2. **Aplicar conocimientos de bases de datos y APIs**
   - Crear y gestionar una base de datos en SQL Server.
   - Diseñar una tabla de productos y poblarla con datos iniciales.
   - Exponer la información mediante un servicio REST en .NET Core.
3. **Desarrollar un endpoint funcional**
   - Implementar controladores en ASP.NET Core para operaciones básicas (GET, POST).
   - Configurar correctamente el contexto de Entity Framework Core.
4. **Consumir un servicio desde otra tecnología**
   - Conectar un cliente PHP al endpoint creado.
   - Mostrar los datos en una interfaz sencilla con estilos intrusivos.
5. **Fomentar la integración tecnológica**
   - Demostrar cómo distintos lenguajes y plataformas (SQL Server, .NET, PHP) pueden trabajar juntos en un sistema distribuido.
   - Resaltar la importancia de la interoperabilidad en arquitecturas modernas.
6. **Desarrollar habilidades prácticas**
   - Reforzar la capacidad de configurar proyectos en Visual Studio.
   - Practicar el consumo de APIs desde clientes externos.
   - Preparar documentación y presentaciones que expliquen el proceso completo.

INTRODUCCIÓN TEÓRICA

Los **microservicios** son un estilo de arquitectura de software que divide una aplicación en servicios pequeños, autónomos y especializados. Cada microservicio se centra en una única responsabilidad y se comunica con otros mediante protocolos ligeros como **HTTP/REST** o **mensajería**.

Características principales:

- **Independencia**: cada servicio se desarrolla, despliega y escala de forma separada.
- **Responsabilidad única**: cada servicio cumple una función específica (ej. productos, usuarios, pedidos).
- **Comunicación ligera**: APIs REST, gRPC o colas de mensajes.
- **Flexibilidad tecnológica**: cada servicio puede implementarse con diferentes lenguajes o frameworks.
- **Resiliencia**: si un servicio falla, los demás pueden seguir funcionando.

En contraste con los **monolitos**, donde toda la lógica está empaquetada en una sola aplicación, los microservicios permiten mayor **escalabilidad**, **mantenimiento más sencillo** y **evolución constante**.

REALIZACIÓN DE PRÁCTICA

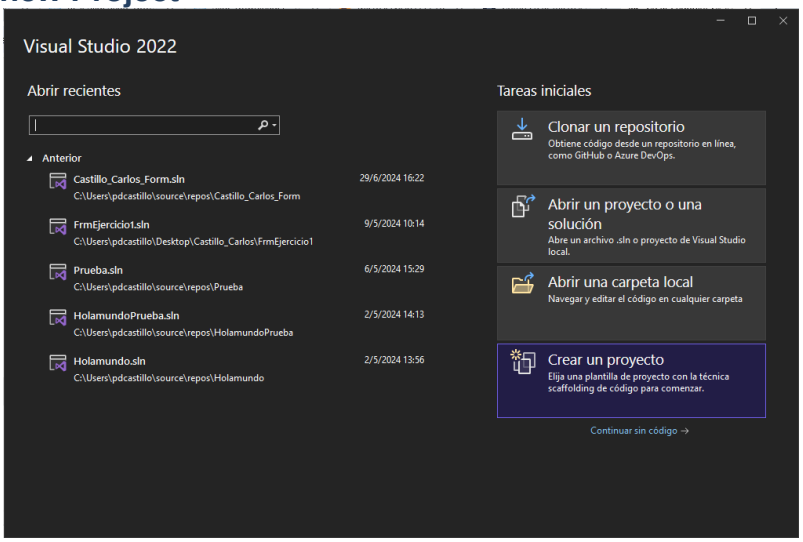
Crearemos nuestra base de datos en SQL Server llamada “DemoApiDb”

```sql
CREATE DATABASE DemoApiDb;
GO
USE DemoApiDb;
GO
CREATE TABLE Products (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(100) NOT NULL,
    Price DECIMAL(10,2) NOT NULL
);
GO
INSERT INTO Products (Name, Price)
VALUES
('Laptop', 1200.00),
('Mouse', 25.00),
('Teclado', 45.00);
GO
```

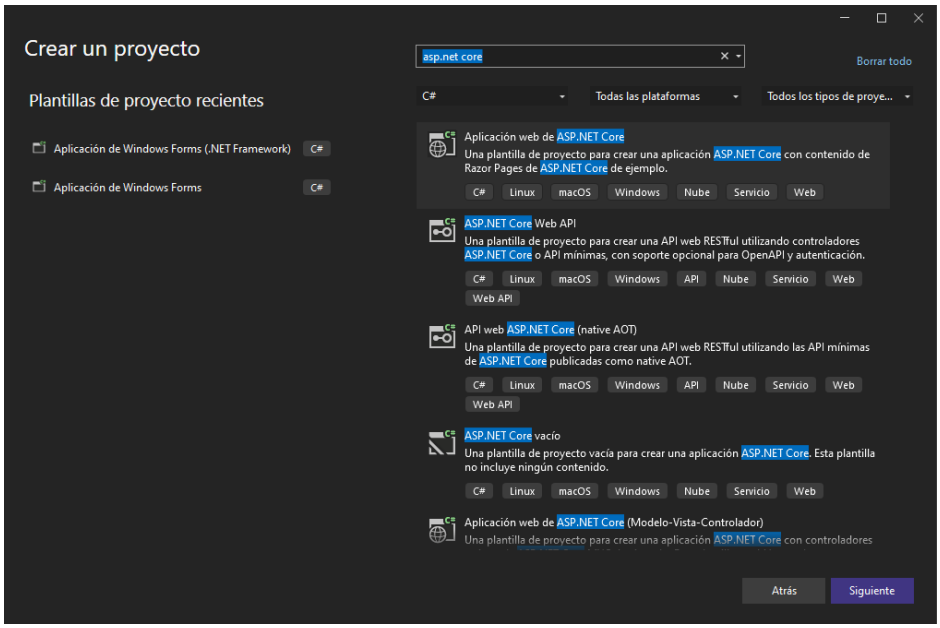
Ahora crearemos nuestro proyecto en Visual Studio con las siguientes características:

PASO 1: Crear el proyecto

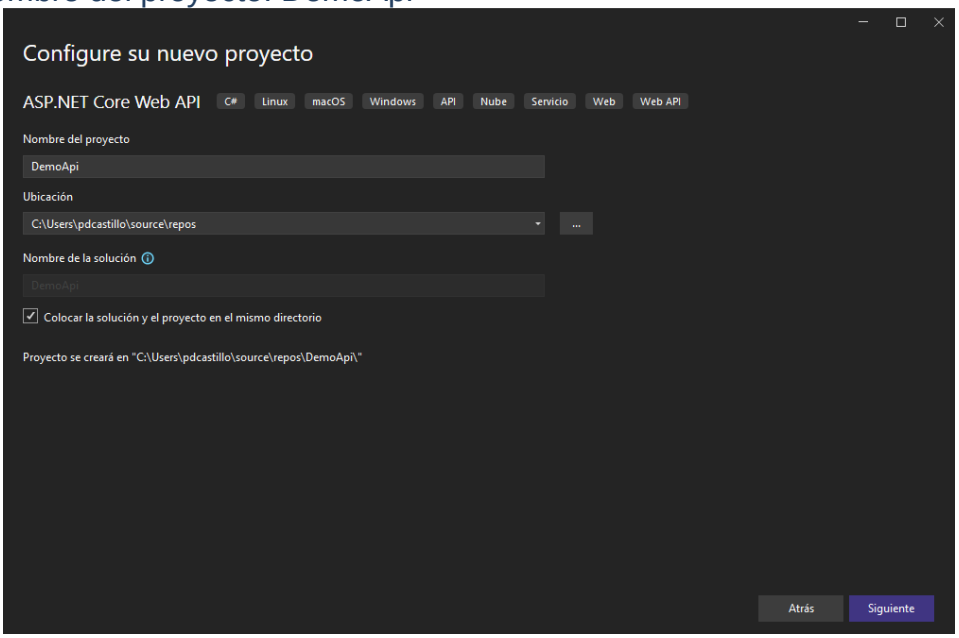
1. Abre **Visual Studio 2022**
2. **Create a new Project**



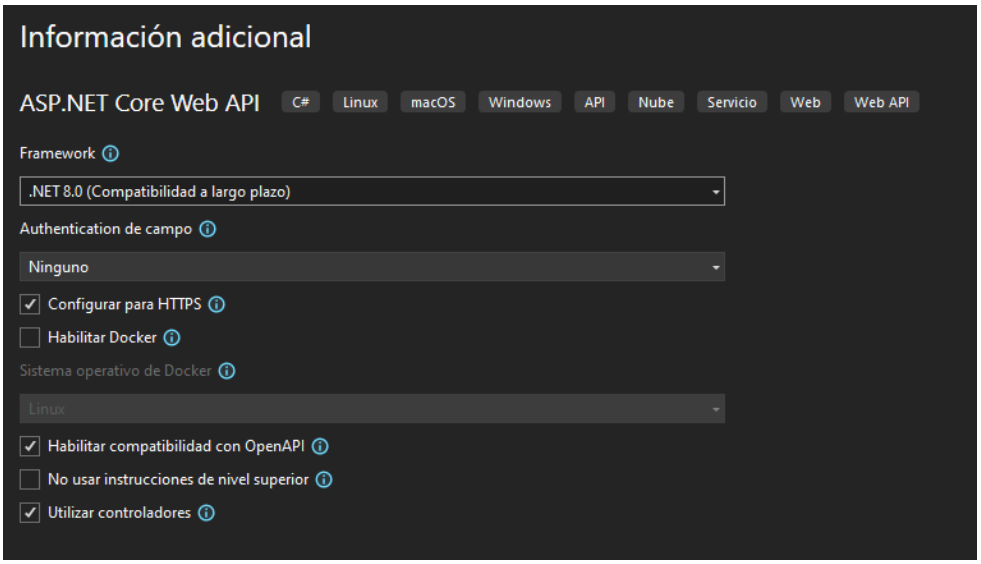
3. Selecciona **ASP.NET Core Web API**



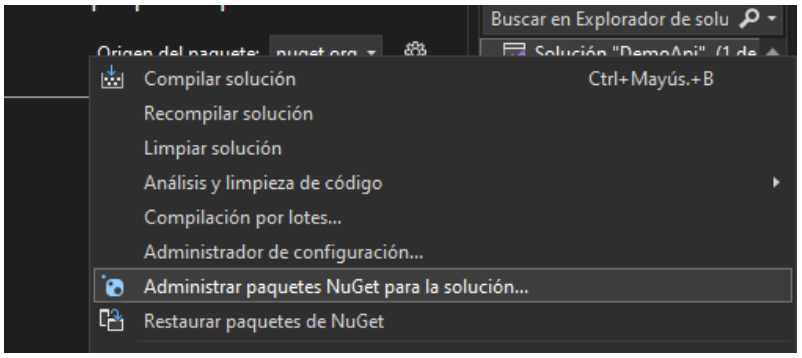
4. Nombre del proyecto: DemoApi



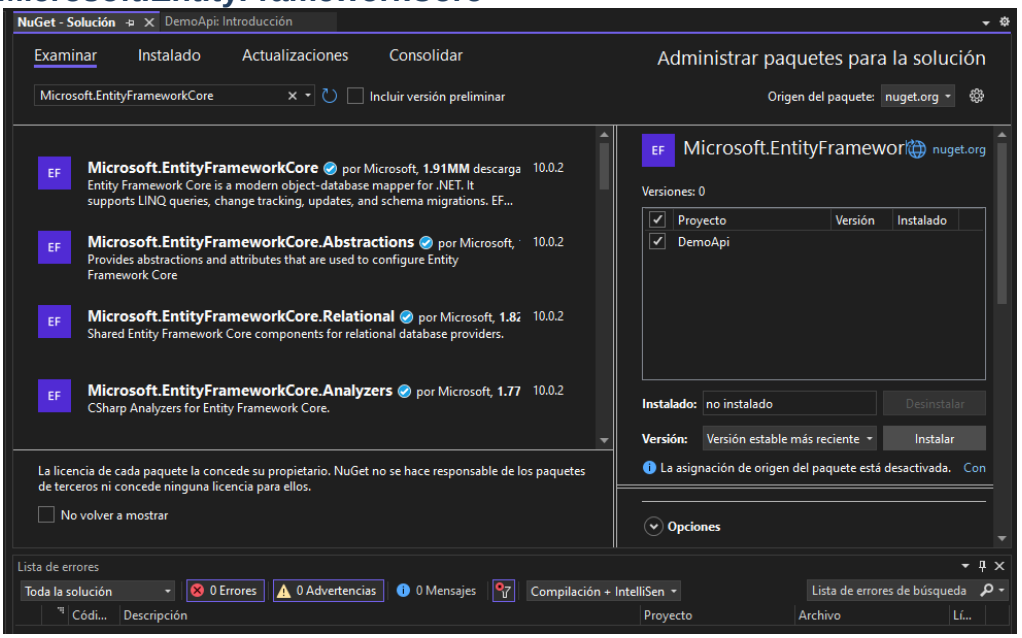
5. Framework: **.NET 7 o .NET 8** (cualquiera está bien)
6. Configuración:
    - Para esta guía usaremos **Utilizar Controladores**
    - Autenticación: **Ninguno**
    - HTTPS: opcional (en este caso si se dejó marcado)
    - Crea el proyecto



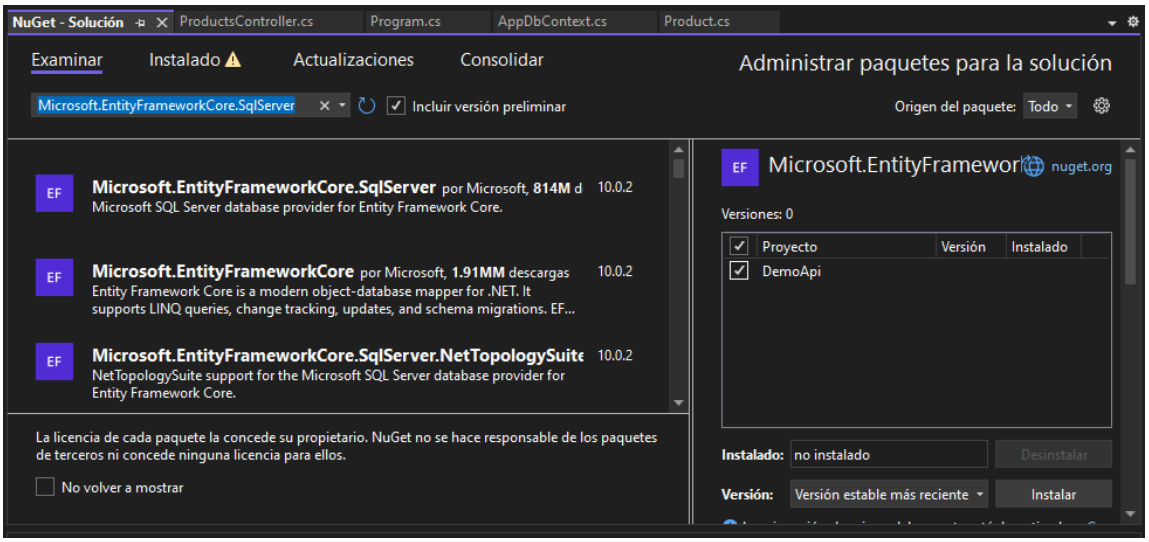
PASO 2: Instalar paquetes necesarios
En **Solution Explorer → click derecho al proyecto → Manage NuGet Packages**
Instala **NOTA PARA TODOS LOS NuGet PACKAGES UTILIZAR VERSION 8.0.XX**:



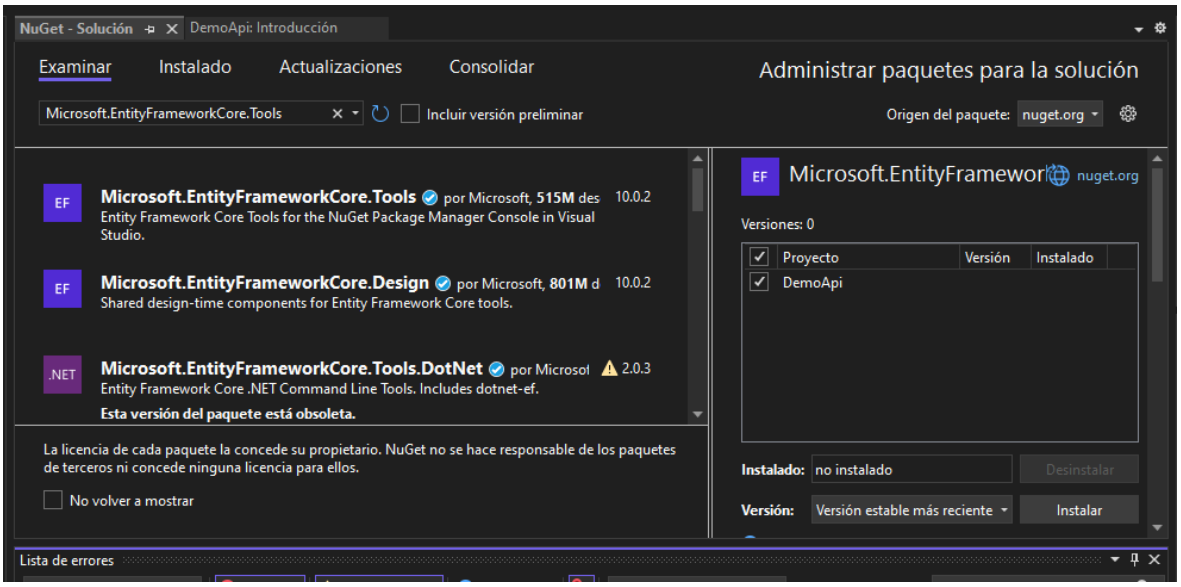
1. **Microsoft.EntityFrameworkCore**



2. **Microsoft.EntityFrameworkCore.SqlServer**



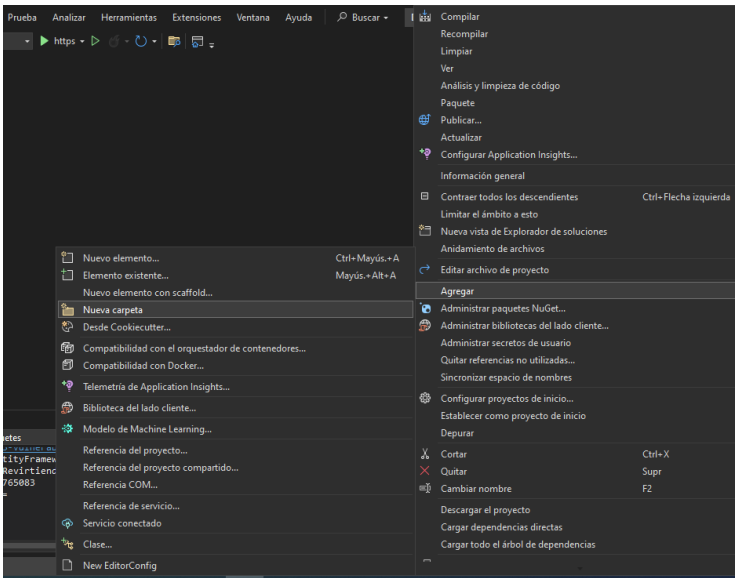
3. **Microsoft.EntityFrameworkCore.Tools**



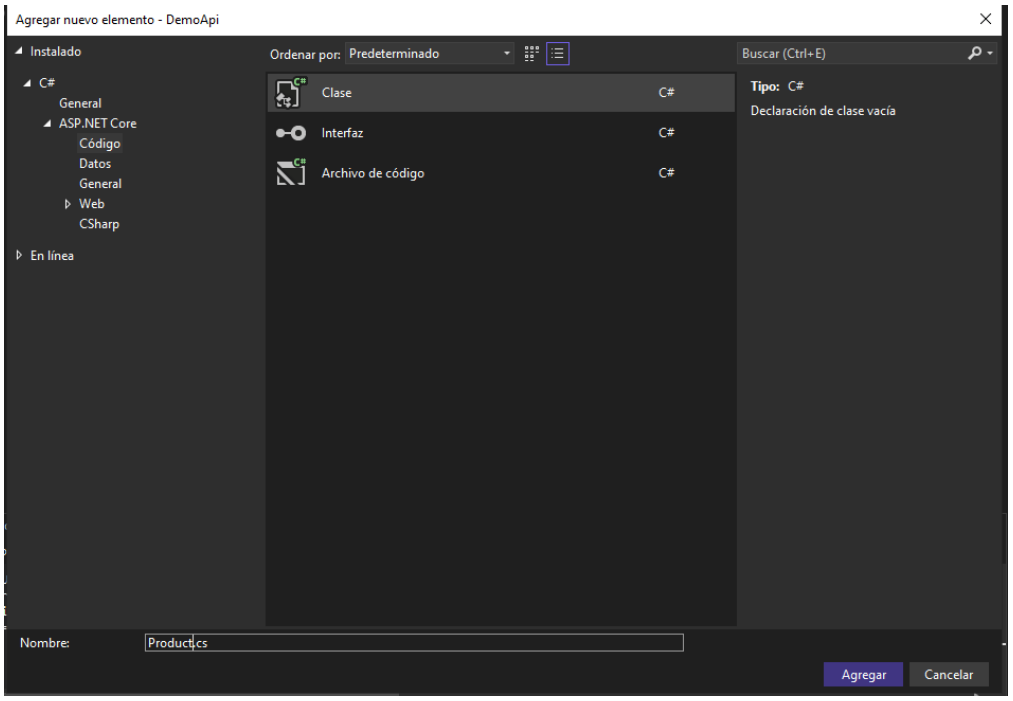
Ahora vamos a configurar la conexión a nuestra base de datos en el archivo appsettings.json
Vamos a agregar dentro del contexto el siguiente bloque de código definiendo con los datos de nuestro servidor SQL Server.

```
"ConnectionStrings": {
  "DefaultConnection": "Server=LHP77\\ARRUPEDB;Database=DemoApiDb;Trusted_Connection=True;TrustServerCertificate=True;"
},
```

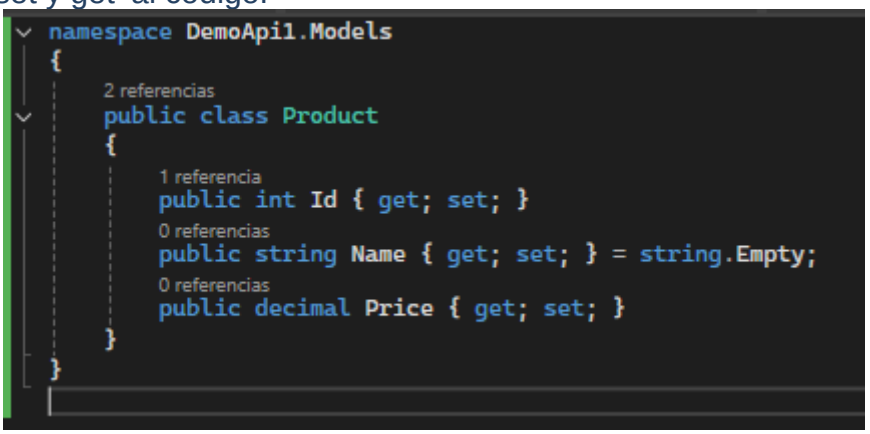
PASO 3: Crear el modelo (Entidad)
Crea una carpeta llamada **Models**



Y una clase llamada Product.cs



Agrega los ‘set y get’ al código:



```
namespace DemoApi1.Models
{
    2 referencias
    public class Product
    {
        1 referencia
        public int Id { get; set; }
        0 referencias
        public string Name { get; set; } = string.Empty;
        0 referencias
        public decimal Price { get; set; }
    }
}
```

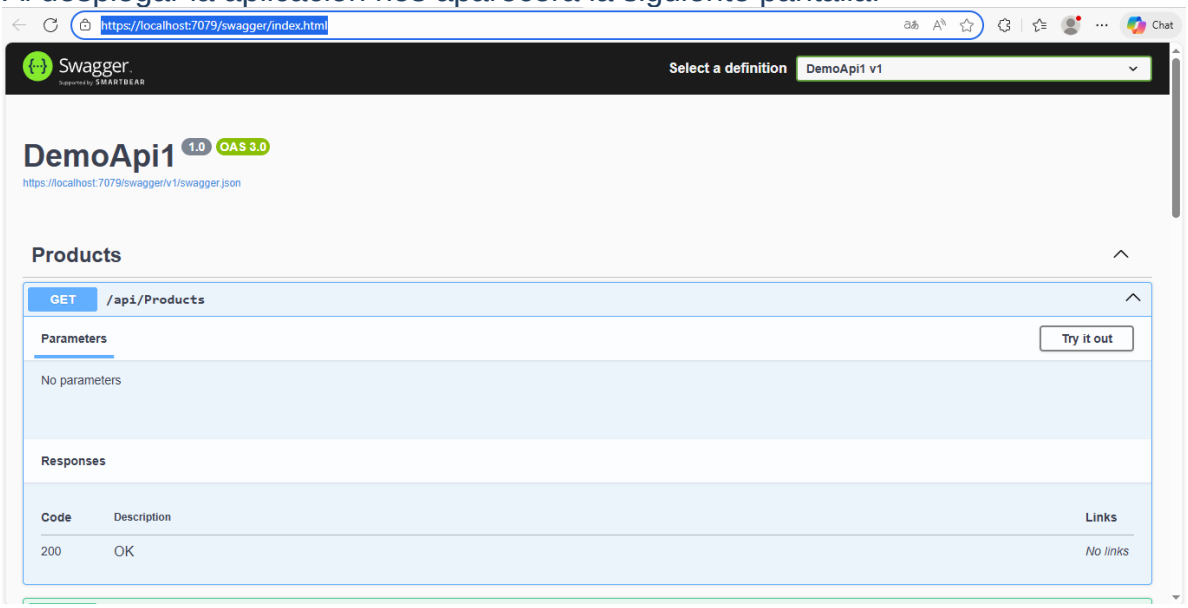
Ahora igual al paso anterior crearemos una carpeta llamada Data y dentro de ella la clase DemoApiDbContext.cs y agregas el siguiente código:

```
using DemoApi1.Models;
using Microsoft.EntityFrameworkCore;

namespace DemoApi1.Data
{
    5 referencias
    public class DemoApiDbContext : DbContext
    {
        0 referencias
        public DemoApiDbContext(DbContextOptions<DemoApiDbContext> options) : base(options)
        {
        }

        3 referencias
        public DbSet<Product> Products { get; set; }
    }
}
```

Ahora en la carpeta Controllers (Creada por defecto en los primeros pasos de config.) vamos a crear la clase ProductsController.cs que es el encargado de tener nuestro endpoint (API):

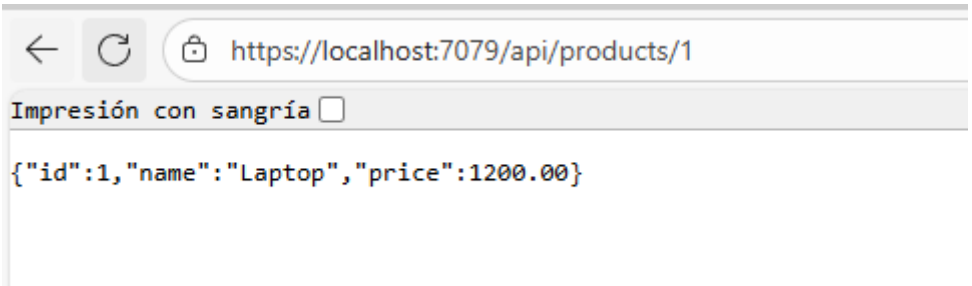
```
using Microsoft.AspNetCore.Mvc;
using DemoApi1.Data;
using DemoApi1.Models;

namespace DemoApi1.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    1 referencia
    public class ProductsController : ControllerBase
    {
        private readonly DemoApiDbContext _context;

        0 referencias
        public ProductsController(DemoApiDbContext context)
        {
            _context = context;
        }

        // GET: api/products
        [HttpGet]
        0 referencias
        public IActionResult GetProducts()
        {
            var products = _context.Products.ToList();
            return Ok(products);
        }

        // GET: api/products/1
        [HttpGet("{id}")]
        1 referencia
        public IActionResult GetProduct(int id)
        {
            var product = _context.Products.Find(id);
            if (product == null) return NotFound();
            return Ok(product);
        }

        // POST: api/products
        [HttpPost]
        0 referencias
        public IActionResult CreateProduct(Product product)
        {
            _context.Products.Add(product);
            _context.SaveChanges();
            return CreatedAtAction(nameof(GetProduct), new { id = product.Id }, product);
        }
    }
}
```

Al desplegar la aplicación nos aparecerá la siguiente pantalla:



Sin embargo para ver nuestro endpoint, ya que no hemos configurado un “index” utilizaremos la siguiente url localhost:7079/api/products así obtendremos acceso a nuestra API

También podemos buscar por id según insertamos en la bdd y configuramos en el controlador agregando a la url el id del producto a buscar: localhost:7079/api/products/1



Vamos a probar el nuestra primer API :D

Crearemos un pequeño ejemplo en php, puedes llamarlo ApiDemo.php tiene que tener la siguiente estructura:

```php
<?php
// URL del endpoint en tu API .NET
$apiUrl = "http://localhost:5107/api/products";
// Consumir el endpoint con file_get_contents (simple)
$response = file_get_contents($apiUrl);

// Decodificar JSON
$products = json_decode($response, true);
?>
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Consumo de API .NET en PHP</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background: linear-gradient(135deg, #1e3c72, #2a5298);
            color: #fff;
            margin: 0;
            padding: 20px;
        }
        h1 {
            text-align: center;
            text-shadow: 2px 2px 5px #000;
        }
        table {
            width: 80%;
            margin: 30px auto;
            border-collapse: collapse;
            background: rgba(255,255,255,0.1);
            box-shadow: 0 0 15px rgba(0,0,0,0.5);
        }
        th, td {
            padding: 12px;
            border: 1px solid #fff;
            text-align: center;
        }
        th {
            background: rgba(0,0,0,0.3);
        }
        tr:hover {
            background: rgba(255,255,255,0.2);
            cursor: pointer;
        }
        .footer {
            text-align: center;
            margin-top: 40px;
            font-size: 14px;
            opacity: 0.7;
        }
    </style>
</head>
<body>
    <h1>Lista de Productos desde API .NET</h1>
    <table>
        <tr>
            <th>ID</th>
            <th>Nombre</th>
            <th>Precio</th>
        </tr>
        <?php if (!empty($products)): ?>
            <?php foreach ($products as $p): ?>
                <tr onclick="alert('Seleccionaste: <?= htmlspecialchars($p['name']) ?>')">
    <td><?= htmlspecialchars($p['id']) ?></td>
    <td><?= htmlspecialchars($p['name']) ?></td>
    <td>$<?= htmlspecialchars($p['price']) ?></td>
</tr>

            <?php endforeach; ?>
        <?php else: ?>
            <tr><td colspan="3">No se encontraron productos</td></tr>
        <?php endif; ?>
    </table>
    <div class="footer">Demo intrusivo con PHP + API .NET, BTVDS 2026</div>
</body>
</html>
```

Si todo funcionó correctamente al abrir la página, debería consumir la API creada en .NET
Y mostrar un listado de los productos:

ACTIVIDAD COMPLEMENTARIA

1. Modificar la Base de datos, para almacenar la imagen del producto (puedes utilizar url) y agregarla al endpoint y mostrarla en la página de prueba .php
2. Para reforzar el aprendizaje, vamos a extender el ejercicio inicial de la API DemoApi con un nuevo microservicio:
   Complemento: Servicio de Usuarios
   - **Crear nueva tabla en SQL Server**
   - **Agregar modelo** User.cs **en .NET**
   - **Actualizar el contexto** DemoApiDbContext
   - **Crear** UsersController.cs con endpoints GET, POST.
   - **Consumir desde PHP** en un archivo separado (users.php) mostrando la lista de usuarios.

BIBLIOGRAFÍA

- Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
- Richardson, C. (2018). *Microservices Patterns: With examples in Java*. Manning Publications.
- Microsoft Docs: Microservices architecture (learn.microsoft.com in Bing)
- Fowler, M. (2014). *Microservices* (artículo en línea). Disponible en: martinfowler.com (martinfowler.com in Bing)
- Dragoni, N., et al. (2017). *Microservices: Yesterday, Today, and Tomorrow*. En *Present and Ulterior Software Engineering*. Springer.